Минимакс и деревья игр

Минимакс и деревья игр

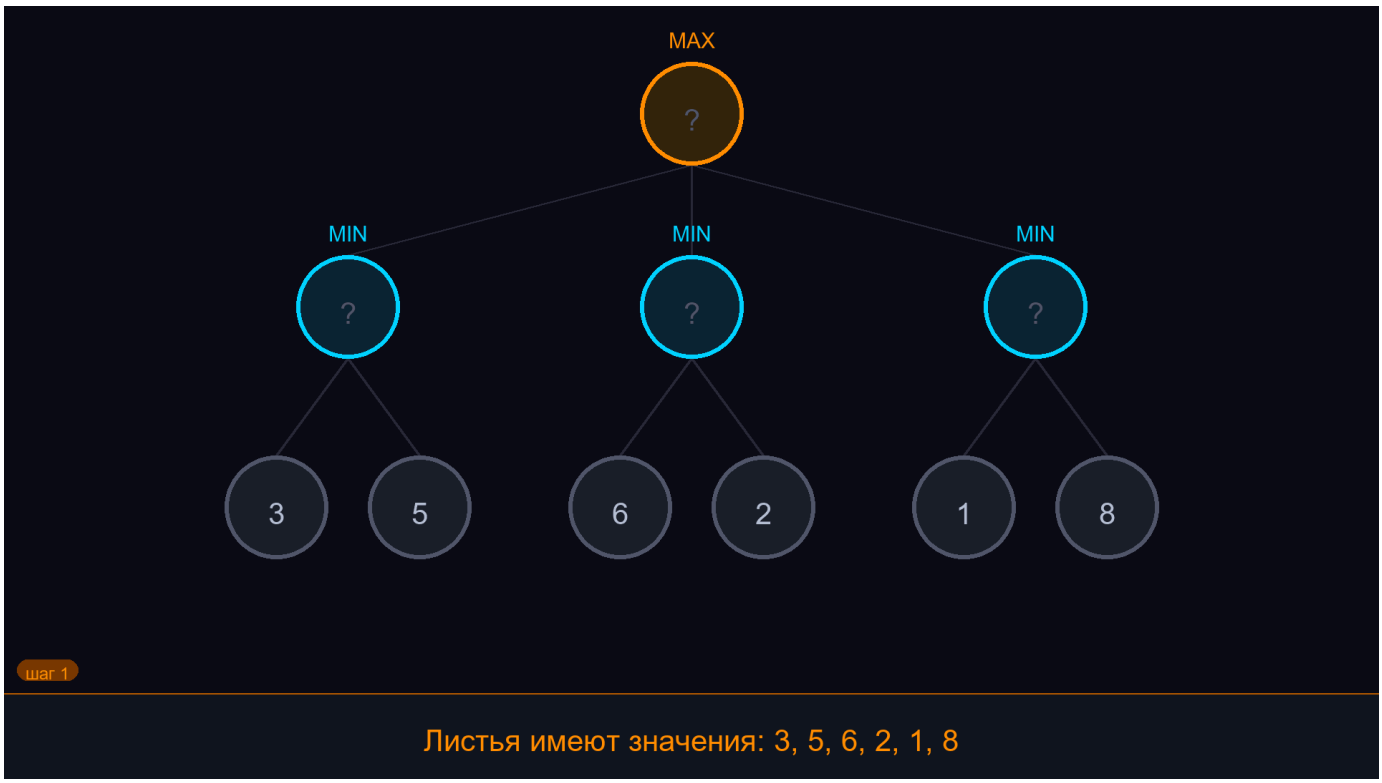
Проблема

В разделе про ИИ врагов мы разбирали поведение в реальном времени: патрулирование, преследование, атака. Но что, если в вашей игре есть пошаговый бой? Или босс, который должен выбирать между атакой, защитой и лечением? Или мини-игра внутри основной игры - карты в таверне, головоломка-замок, шахматный турнир у NPC?

Ну, наивный подход: ИИ босса делает случайное действие. Атака, лечение, атака, атака, лечение при полном здоровье... Играть против такого противника неинтересно - он не сопротивляется, не адаптируется, не наказывает за ошибки. Можно захардкодить правила ("если здоровье < 30% - лечиться"), но правил быстро становится много, они конфликтуют, и босс всё равно ведёт себя предсказуемо.

А что, если босс сам найдёт лучший ход? Не потому что мы запрограммировали конкретное правило, а потому что он **просчитал варианты** и выбрал тот, который ведёт к победе (или хотя бы не к поражению)?

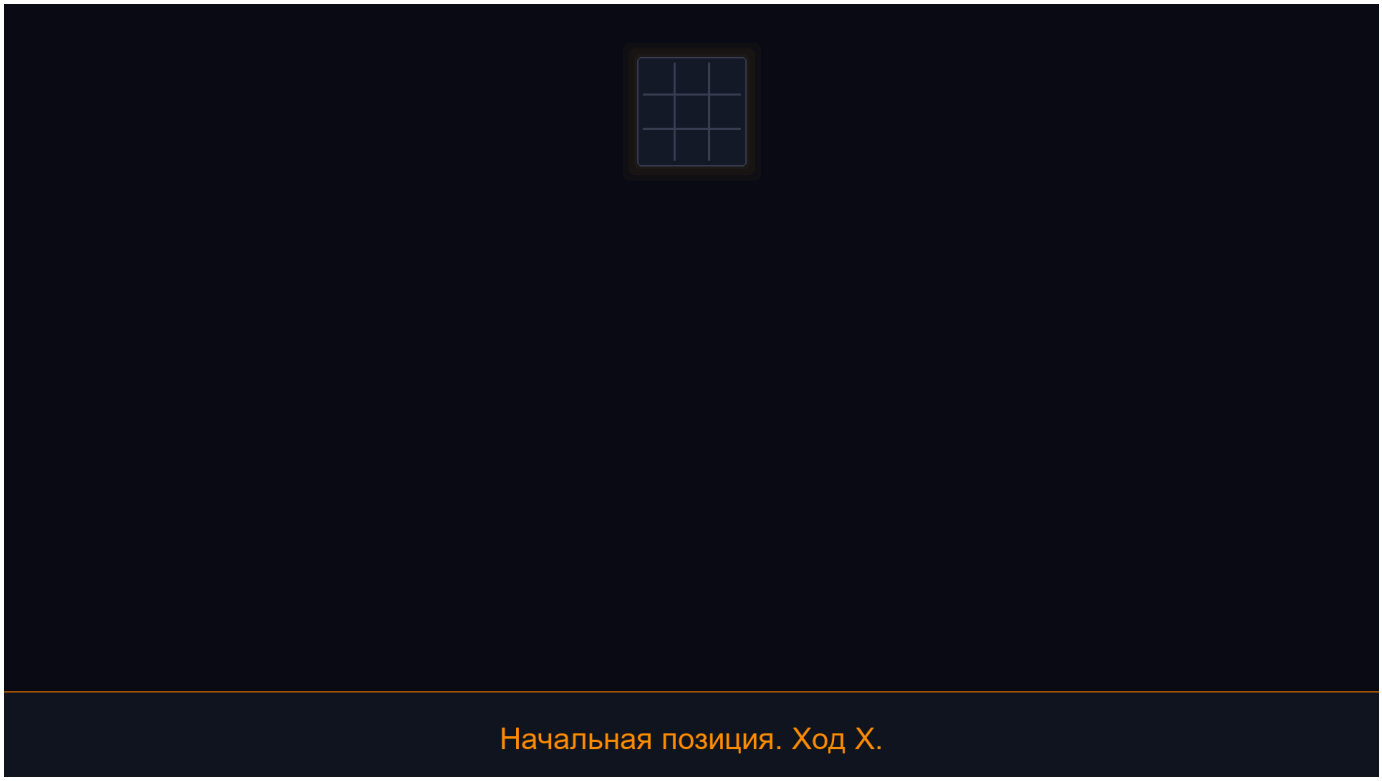
Именно это делает алгоритм **минимакс** (minimax).



Дерево игры

Допустим, мы играем в крестики-нолики. Сейчас ход X. У X есть 9 возможных ходов (9 пустых клеток). После каждого хода X у O будет 8 возможных ходов. После каждого хода O у X будет 7. И так далее.

Если нарисовать все возможные последовательности ходов, получится **дерево** - та же структура, что граф из раздела про уровни и поиск пути, только здесь вершины - это состояния доски, а рёбра - ходы.



Полное дерево крестиков-ноликов содержит ~255 000 вершин. Это немного - компьютер перебирает их за миллисекунды. Для шахмат дерево содержит примерно 10^120 вершин - грубо говоря, это больше, чем атомов во Вселенной. Полный перебор невозможен, но идея та же.

Каждый **лист** дерева (конечная позиция) - это результат: X выиграл, O выиграл, или ничья. Зная результаты всех возможных окончаний, можно выбрать ход, который гарантированно ведёт к лучшему исходу.

Идея минимакса

В пошаговой игре двух игроков один хочет **максимизировать** свой результат, другой - **минимизировать** результат противника (т.е. максимизировать свой). Мы называем их:

- **Максимизатор** (MAX) - ИИ, наш компьютерный игрок. Выбирает ход с наибольшей оценкой.
- **Минимизатор** (MIN) - противник. Выбирает ход с наименьшей оценкой (для MAX).

Ключевое допущение: мы считаем, что противник играет **оптимально** - всегда выбирает лучший для себя ход. Если ИИ может выиграть даже против оптимального противника - он выиграет и против слабого.

Алгоритм:

1. Строим дерево всех возможных ходов до конца игры.
2. Листьям присваиваем оценку: победа MAX = **+10**, победа MIN = **-10**, ничья = **0**.

3. Поднимаемся по дереву снизу вверх:
   - Если это ход MAX - выбираем **максимум** из дочерних узлов.
   - Если это ход MIN - выбираем **минимум** из дочерних узлов.
4. В корне получаем оценку лучшего хода для MAX.

Анимация в начале страницы показывает этот процесс: оценки поднимаются снизу вверх, MAX берёт максимум, MIN - минимум. Отсюда и название: **мини**мум-**макс**имум.

Оценочная функция

Чтобы минимакс работал, нужна **оценочная функция** (evaluation function) - способ присвоить числовую оценку позиции.

Для крестиков-ноликов всё просто - игра всегда доходит до конца:

Python C#

```python
def evaluate(board, ai_mark, opponent_mark):
    winner = check_winner(board)
    if winner == ai_mark:
        return 10
    elif winner == opponent_mark:
        return -10
    else:
        return 0  # draw or game not over
```

Для более сложных игр (шахматы, шашки) оценочная функция - это эвристика: "стоимость фигур + контроль центра + безопасность короля + ...". Чем точнее эвристика, тем сильнее ИИ. Это один из ключевых моментов, где алгоритмы встречаются с предметной областью.

Реализация: крестики-нолики

Для демонстрации возьмём крестики-нолики - дерево маленькое, правила очевидны, результат можно проверить руками. Доска - массив 3x3, ходы - X и O.

Нам понадобятся две вспомогательные функции - `check_winner(board)` (проверяет строки, столбцы и диагонали) и `get_empty_cells(board)` (возвращает список пустых клеток). Их реализация прямолинейная - 8 линий для проверки, цикл по ячейкам. Самое интересное - рекурсия:

Python C#

```python
def minimax(board, is_maximizing, ai_mark, opp_mark):
    winner = check_winner(board)
```

```
    if winner == ai_mark: return 10
    if winner == opp_mark: return -10
    if not get_empty_cells(board): return 0  # draw

    if is_maximizing:
        best = -100
        for r, c in get_empty_cells(board):
            board[r][c] = ai_mark
            score = minimax(board, False, ai_mark, opp_mark)
            board[r][c] = " "           # undo move!
            best = max(best, score)
        return best
    else:
        best = 100
        for r, c in get_empty_cells(board):
            board[r][c] = opp_mark
            score = minimax(board, True, ai_mark, opp_mark)
            board[r][c] = " "
            best = min(best, score)
        return best
```

`FindBestMove` - обёртка: перебирает все пустые клетки, для каждой вызывает `minimax(board, false, ...)` и запоминает ход с лучшей оценкой.

Обратите внимание на рекурсию: `minimax` вызывает сам себя, чередуя `is_maximizing`. На каждом уровне мы делаем ход, оцениваем, **отменяем ход** (`board[r][c] = " "`). Это и есть обход дерева игры в глубину - по сути, тот же DFS, только с оценкой. Против такого ИИ в крестиках-ноликах невозможно выиграть - максимум ничья.

Ключевой приём - "сделай ход, рекурсия, отмени ход". Без отмены доска испортится и следующие ветки дерева будут считаться по неправильному состоянию.

Проблема: слишком много вариантов

Для крестиков-ноликов полный перебор работает. 9 клеток, максимум 9! = 362 880 вариантов (на практике - поменьше из-за ранних побед). Компьютер справляется мгновенно.

А что если доска 15x15 (гомоку)? Или шахматы, где средняя длина партии ~80 ходов, а на каждом ходу ~30 вариантов? 30^80 - число с 118 нулями. Никакой компьютер это не переберёт.

Нужно как-то сократить перебор, не потеряв качество решения. И тут появляется **альфа-бета отсечение**.

Альфа-бета отсечение

Идея: если мы уже знаем, что ветка дерева не может дать результат лучше, чем то, что мы уже нашли, - зачем её проверять?

Пример: MAX нашёл ход с оценкой 5. Теперь проверяем другую ветку. В этой ветке MIN уже нашёл ответ с оценкой 3. Т.е. в лучшем случае MAX получит 3 из этой ветки. Но у нас уже есть 5! Значит, остальные варианты в этой ветке можно не проверять - они бесполезны.

Два параметра:

- **alpha** - лучший (максимальный) результат, который MAX уже гарантировал себе.
- **beta** - лучший (минимальный) результат, который MIN уже гарантировал себе.

Если alpha >= beta - отсекаем ветку.

В код минимакса добавляются два параметра (`alpha` , `beta`) и по три строки в каждую ветку. Вот что меняется внутри цикла перебора ходов:

```
// in MAX branch (after computing score):
alpha = max(alpha, best)
if alpha >= beta:
    break          // MIN won't pick this path - prune

// in MIN branch (after computing score):
beta = min(beta, best)
if alpha >= beta:
    break          // MAX already found a better option - prune

// initial call: alpha = -∞, beta = +∞
score = minimax_ab(board, True, "O", "X", -100, 100)
```

Результат **тот же**, что и у обычного минимакса - оптимальный ход. Но вычислений меньше. Для крестиков-ноликов альфа-бета сокращает перебор в несколько раз, а при хорошем порядке ходов в шахматах - на порядки.

Альфа-бета отсечение - не отдельный алгоритм, а **оптимизация** минимакса. Оно не меняет ответ, только ускоряет поиск. Вся разница - шесть строк кода.

Ограничение глубины

Даже с альфа-бета отсечением шахматы не решить полным перебором. Дерево слишком глубокое. Выход: ограничить глубину поиска. Вместо "просчитай до конца игры" - "просчитай на N ходов вперёд".

Но если мы не дошли до конца игры, как оценить позицию? Вот тут и нужна **эвристическая оценочная функция** - не точный результат (победа/поражение), а приблизительная оценка ("эта позиция скорее хорошая для MAX").

К минимаксу добавляется параметр `depth` и одна проверка в начале:

```
def minimax_depth(board, depth, is_maximizing, ..., alpha, beta):
    # terminal states: win/loss/draw - same as before
    ...

    if depth == 0:
        return heuristic_evaluate(board)   # <-- the new part

    # same as before, but recurse with depth - 1:
    score = minimax_depth(board, depth - 1, not is_maximizing, ..., alpha, beta
```

А **эвристическая оценка** - это уже творчество. Для крестиков-ноликов: сколько линий ещё можно выиграть минус сколько может противник. Для шахмат: стоимость фигур + контроль центра + безопасность короля. Чем точнее эвристика, тем сильнее ИИ при той же глубине.

Ок. Глубина - это компромисс. `depth = 2` - ИИ "видит" на 2 хода вперёд. Быстро, но легко обмануть. `depth = 6` - серьёзный противник, но может задуматься на секунду. `depth = 10` - почти непобедим, но на сложных играх будет считать слишком долго.

Именно так работали шахматные программы до появления нейросетей: минимакс + альфа-бета + хорошая оценочная функция + ограничение глубины. Deep Blue (1997) просчитывал на 12-20 ходов вперёд. Разница между "средним" и "великим" шахматным ИИ часто сводилась к качеству оценочной функции, а не к глубине поиска.

Настройка сложности

С минимаксом легко сделать разные уровни сложности - и это не костыль, а прямое следствие архитектуры:

Уровень	Как реализовать
Лёгкий	Случайный ход (без минимакса)
Средний	Минимакс с малой глубиной (2-3)
Сложный	Минимакс с большой глубиной (6+) или без ограничения
"Почти идеальный"	Полный минимакс, но иногда (10%) - случайный ход

Последний вариант интересен: ИИ играет оптимально, но иногда делает "ошибку". Это создаёт ощущение, что противник живой, а не машина. Игрок чувствует, что может выиграть, но должен стараться.

Пример: пошаговый бой с боссом

Крестики-нолики - хорошая классика, но давайте применим минимакс к чему-то, что ближе к реальности. Допустим, у нас пошаговая RPG. Игрок и босс по очереди выбирают действие. У каждого есть очки здоровья (HP), и набор действий:

- **Атака** - наносит урон противнику.
- **Лечение** - восстанавливает здоровье себе.
- **Сильный удар** - наносит большой урон, но пропускает следующий ход.

"Доска" здесь - это не сетка, а **состояние боя**: HP игрока, HP босса, чей ход, есть ли активные эффекты. Принцип тот же: перебираем действия, оцениваем результат, выбираем лучшее.

Минимакс - это **каркас**. Для пошагового боя меняются только три вещи - а рекурсивная структура остаётся ровно такой же, как в крестиках-ноликах:

Крестики-нолики	Пошаговый бой
Доска 3x3	`(player_hp, boss_hp, stunned?, чей_ход)`
Поставить X/O в клетку	`["attack", "heal", "heavy_attack"]`
Кто выстроил линию? ±10	HP ≤ 0? ±100, иначе `boss_hp - player_hp`

```
состояние = {player_hp: 80, boss_hp: 45, boss_turn: true}
действия  = [attack(-15 hp), heal(+20 hp), heavy_attack(-30 hp, skip next)]

оценка:
  boss_hp <= 0  -> -100  (босс проиграл)
  player_hp <= 0 -> +100  (босс выиграл)
  иначе          -> boss_hp - player_hp  (эвристика)
```

Сам минимакс вызывается точно так же - только вместо перебора клеток перебираем действия.

Результат: босс при `depth = 6` просчитывает на 6 ходов вперёд. Если у него 45 HP против 80 у игрока, он "понимает", что при обмене ударами проиграет, и начинает лечиться. Если у игрока мало HP - бьёт сильным ударом, жертвуя следующим ходом, потому что знает, что добьёт.

Та же схема работает для любой пошаговой игры. Карточная дуэль? Состояние = карты на руках + стол. Действия = сыграть карту. Оценка = сила позиции. Мини-игра в таверне, головоломка-дуэль, шахматный турнир у NPC - один и тот же каркас.

От перебора к обучению

Минимакс - это **поиск в состязательной среде** (adversarial search). Если разобраться, внутри - знакомые вещи:

- **DFS по дереву игры.** Тот же обход графа, только с оценкой и двумя игроками.
- **Оценочная функция - это эвристика**, как h(n) в A*. A* оценивает "насколько далеко до цели", минимакс - "насколько хороша позиция".
- **Альфа-бета - отсечение пространства поиска**, та же идея, что в пространственных структурах: не проверять то, что точно не нужно.

А дальше - эволюция подхода:

- **Monte Carlo Tree Search (MCTS)** - вместо полного перебора делаем случайные "прогоны" партии до конца и статистически оцениваем, какой ход лучше. Именно так работает AlphaGo.
- **Эволюционные алгоритмы** - вместо дерева перебора поддерживаем *популяцию* кандидатов и улучшаем её через отбор и мутацию. Подробнее: Генетические алгоритмы.
- **Обучение с подкреплением** - ИИ не перебирает дерево, а *учится* предсказывать, какие позиции хорошие. Нейросеть заменяет оценочную функцию. Это AlphaZero.

Т.е. минимакс - фундамент. Перебор -> отсечение -> эвристика -> обучение - это и есть эволюция игрового ИИ.

Итого

Концепция	Что делает
Дерево игры	Все возможные последовательности ходов
Минимакс	Полный перебор: MAX выбирает max, MIN выбирает min
Оценочная функция	Числовая оценка позиции (точная или эвристическая)
Альфа-бета	Отсечение бесполезных веток. Тот же ответ, меньше работы
Ограничение глубины	Не перебираем до конца, оцениваем эвристикой

Что	Где почитать подробнее
Графы и поиск пути	Уровни и поиск пути
Обзор алгоритмов, A*	Алгоритмы
ИИ врагов в реальном времени	Простой ИИ врагов
Конечные автоматы для состояний	Конечные автоматы
Событийная модель (ход сделан -> обновить UI)	Событийная модель

Что	Где почитать подробнее
Пошаговый цикл, Command, undo/redo	Пошаговые игры и паттерн Command

Powered by Wiki.js